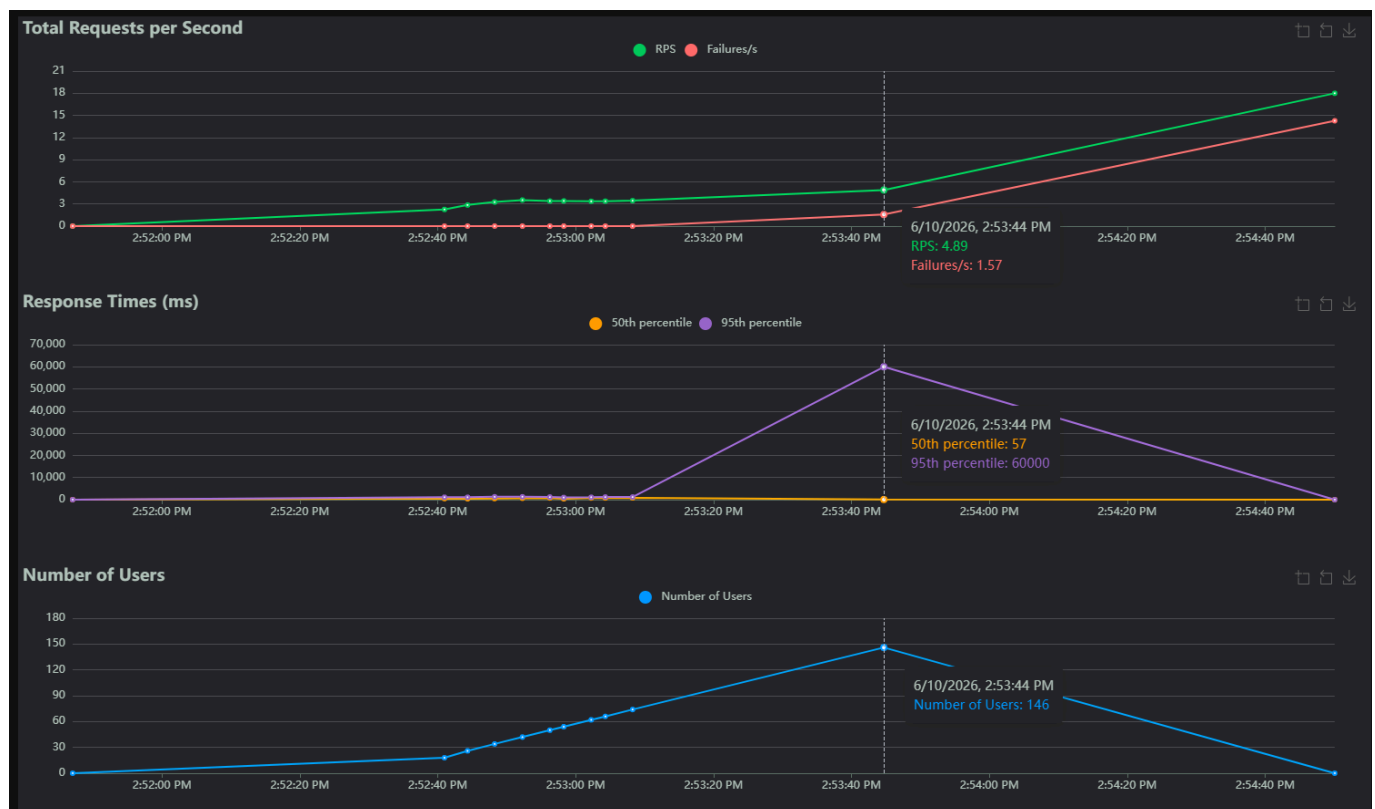


Dyaa Abou Arida

Rapport Labo 4

Question 1 : Combien d'utilisateurs faut-il pour que le Store Manager commence à échouer dans votre environnement de test ? Pour répondre à cette question, comparez la ligne Failures et la ligne Users dans les graphiques.

Le Store Manager commence à échouer autour de 146 utilisateurs simultanés, la ligne des échecs commence à augmenter de façon visible.



1.1 Capture d'écran de Charts

Question 2 : Sur l'onglet Statistics, comparez la différence entre les requêtes et les échecs pour tous les endpoints. Combien d'entre eux échouent plus de 50 % du temps ?

Le endpoint POST /orders échoue 100% du temps, tandis que les deux endpoints de rapports échouent environ 67% du temps.

STATISTICS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders	776	776	42	60000	60000	8970.99	0	60061	105.75	16.2	16.2
GET	/orders/reports/best-sellers	682	466	690	3100	3500	914.09	35	3817	1410.96	12.7	9.7
GET	/orders/reports/highest-spenders	703	472	390	1700	2100	599.18	31	2317	315.44	13.1	11.7
Aggregated		2161	1714	340	31000	60000	3704.82	0	60061	585.88	42	37.6

2.1 Capture d'écran des statistiques

Question 3 : Affichez quelques exemples des messages d'erreur affichés dans l'onglet Failures. Ces messages indiquent une défaillance dans quelle(s) partie(s) du Store Manager ? Par exemple, est-ce que le problème vient du service Python / MySQL / Redis / autre ?

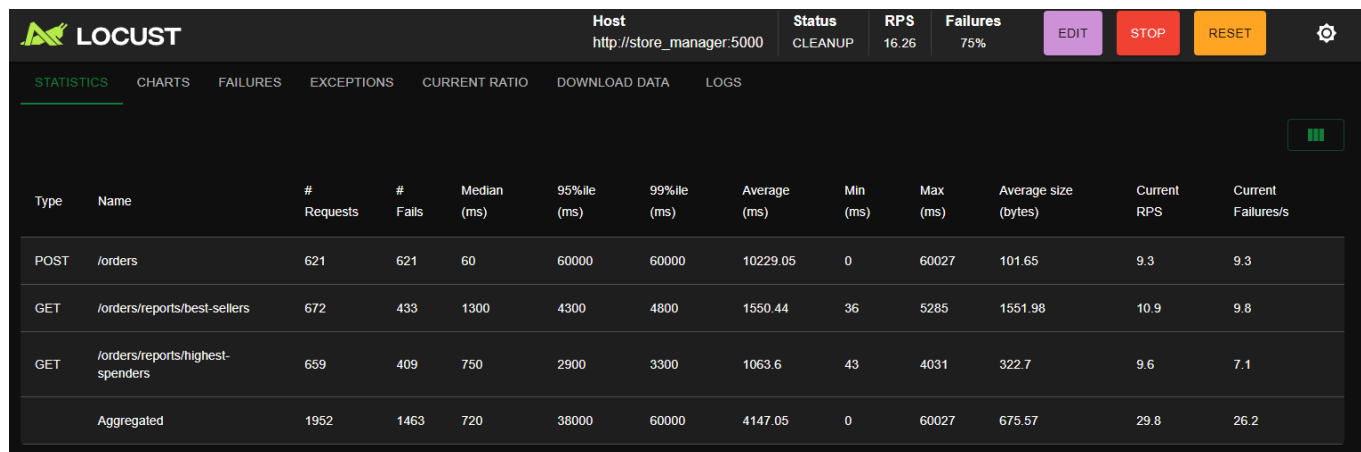
Les erreurs indiquent principalement une défaillance au niveau de MySQL. On observe des erreurs Too many connections, ce qui signifie que la base de données reçoit trop de connexions simultanées.

STATISTICS						
CHARTS						
FAILURES						
EXCEPTIONS						
CURRENT RATIO						
DOWNLOAD DATA						
LOGS						
# Failures	Method	Name	Message	First Seen	Last Seen	
78	POST	/orders	Exception("Aucune réponse (erreur ou timeout)")	6/10/2026, 2:53:33 PM	6/10/2026, 2:54:32 PM	
78	POST	/orders	RetriesExceeded('http://store_manager:5000/orders', 0, original=timed out)	6/10/2026, 2:53:33 PM	6/10/2026, 2:54:32 PM	
287	POST	/orders	CatchResponseError("Erreur : 500 - (mysql.connector.errors.OperationalError) 1040 (08004): Too many connections\n(Background on this error at: https://sqlalche.me/e/20/e3q8)")	6/10/2026, 2:53:37 PM	6/10/2026, 2:54:31 PM	
238	POST	/orders	CatchResponseError("Erreur : 500 - (mysql.connector.errors.DatabaseError) 1040 (HY000): Too many connections\n(Background on this error at: https://sqlalche.me/e/20/4xp6)")	6/10/2026, 2:53:39 PM	6/10/2026, 2:54:32 PM	
94	POST	/orders	CatchResponseError("Erreur : 500 - Too many connections")	6/10/2026, 2:53:44 PM	6/10/2026, 2:54:32 PM	
1	POST	/orders	CatchResponseError("Erreur : 500 - (mysql.connector.errors.InternalError) 1213 (40001): Deadlock found when trying to get lock; try restarting transaction\n[SQL: 'UPDATE stocks SET quantity = quantity - %(qty)s WHERE product_id = %(pid)s'\n]\n[parameters: {'qty': 1, 'pid': 1}]\n(Background on this error at: https://sqlalche.me/e/20/2j85)")	6/10/2026, 2:54:13 PM	6/10/2026, 2:54:13 PM	
466	GET	/orders/reports/best-sellers	CatchResponseError("Erreur : 500 - Aucun JSON dans la réponse. Message : <doctype html>\n<html lang=en>\n<title>500 Internal Server Error</title>\n Internal Server Error \n The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application. \n")	6/10/2026, 2:53:37 PM	6/10/2026, 2:54:32 PM	
472	GET	/orders/reports/highest-spenders	CatchResponseError("Erreur : 500 - Aucun JSON dans la réponse. Message : <doctype html>\n<html lang=en>\n<title>500 Internal Server Error</title>\n Internal Server Error \n The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application. \n")	6/10/2026, 2:53:38 PM	6/10/2026, 2:54:32 PM	

3.1 Capture d'écran des failures

Question 4 : Sur l'onglet Statistics, comparez les résultats actuels avec les résultats du test de charge précédent. Est-ce que vous voyez quelques différences dans les métriques pour l'endpoint POST /orders ?

Après l'optimisation du problème N+1, je ne constate pas d'amélioration significative pour l'endpoint POST /orders. L'endpoint continue d'échouer 100% du temps. Les métriques montrent que le temps de réponse reste très élevé. Cela indique que le problème principal ne vient pas uniquement de la récupération des prix des produits, mais surtout de la surcharge de MySQL lors des écritures simultanées.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders	621	621	60	60000	60000	10229.05	0	60027	101.65	9.3	9.3
GET	/orders/reports/best-sellers	672	433	1300	4300	4800	1550.44	36	5285	1551.98	10.9	9.8
GET	/orders/reports/highest-spenders	659	409	750	2900	3300	1063.6	43	4031	322.7	9.6	7.1
Aggregated		1952	1463	720	38000	60000	4147.05	0	60027	675.57	29.8	26.2

4.1 Capture d'écran des statistiques

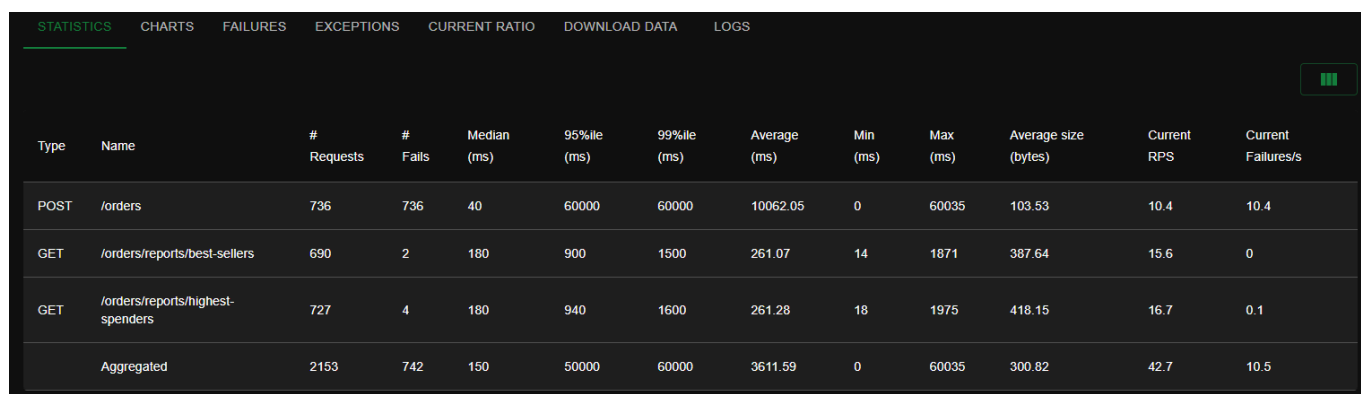
Question 5 : Si nous avons plus d'articles dans notre base de données (par exemple, 1 million), ou simplement plus d'articles par commande en moyenne, le temps de réponse de l'endpoint POST /orders augmenterait-il, diminuerait-il ou resterait-il identique ?

Si la base de données contenait plus d'articles le temps de réponse ne devrait pas nécessairement augmenter beaucoup pour chercher les produits par id parce que cette recherche est indexée. Par contre, si chaque commande contenait plus d'articles en moyenne, le temps de réponse de POST /orders augmenterait parce que l'application devrait traiter plus d'items, calculer plus de prix, insérer plus de lignes OrderItem et mettre à jour plus de stocks.

Question 6 : Sur l'onglet Statistics, comparez les résultats actuels avec les résultats du test de charge précédent. Est-ce que vous voyez quelques différences significatives dans les métriques pour les endpoints POST /orders, GET /orders/reports/highest-spenders et GET /orders/reports/best-sellers ? Dans quelle mesure la performance s'est-elle améliorée ou détériorée (par exemple, en pourcentage) ?

Après l'implémentation du cache Redis optimisé, les performances des endpoints de lecture se sont fortement améliorées. Après l'optimisation, best-sellers a seulement 2 échecs sur 690 requêtes, soit environ 0,29%, et highest-spenders a 4 échecs sur 727 requêtes, soit environ 0,55%. Les deux endpoints de rapports deviennent donc presque entièrement disponibles sous charge.

Par contre, l'endpoint POST /orders ne s'améliore pas. Il échoue encore 100% du temps, avec 736 échecs sur 736 requêtes. Cela montre que l'optimisation du cache améliore surtout la lecture des rapports, mais ne règle pas les problèmes d'écriture liés à MySQL.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders	736	736	40	60000	60000	10062.05	0	60035	103.53	10.4	10.4
GET	/orders/reports/best-sellers	690	2	180	900	1500	261.07	14	1871	387.64	15.6	0
GET	/orders/reports/highest-spenders	727	4	180	940	1600	261.28	18	1975	418.15	16.7	0.1
Aggregated		2153	742	150	50000	60000	3611.59	0	60035	300.82	42.7	10.5

6.1 Capture d'écran des statistiques

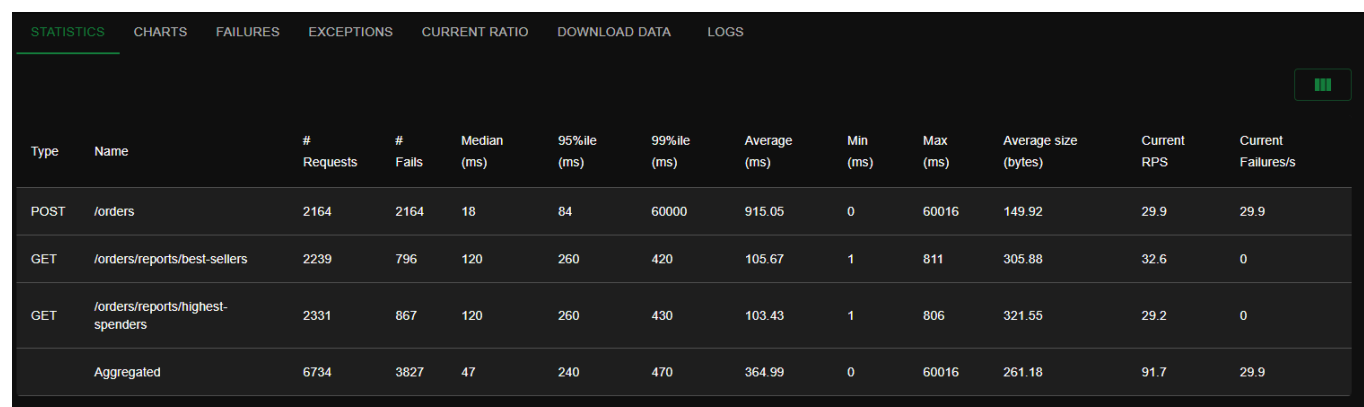
Question 7 : La génération de rapports repose désormais entièrement sur des requêtes adressées à Redis, ce qui réduit la charge pesant sur MySQL. Cependant, le point de terminaison POST /orders reste à la traîne par rapport aux autres en termes de performances dans notre scénario de test. Alors, qu'est-ce qui limite les performances de l'endpoint POST /orders ?

Les performances de POST /orders sont limitées par MySQL. Contrairement aux rapports, la création d'une commande nécessite encore plusieurs écritures synchrones : création de la commande, insertion des items, mise à jour du stock et transaction SQL. Sous forte charge, cela cause trop de connexions. Redis améliore donc les lectures, mais ne règle pas le problème des écritures concurrentes dans MySQL.

Question 8 : Sur l'onglet Statistics, comparez les résultats actuels avec les résultats du test de charge précédent. Est-ce que vous voyez quelques différences significatives dans les métriques pour les endpoints POST /orders, GET /orders/reports/highest-spenders et GET /orders/reports/best-sellers ? Dans quelle mesure la performance s'est-elle améliorée ou détériorée (par exemple, en pourcentage) ? La réponse dépendra de votre environnement d'exécution (par exemple, vous obtiendrez de meilleures performances en exécutant 2 instances de Store Manager sur 2 machines virtuelles plutôt que sur une seule).

Avec le load balancing Nginx, le système traite beaucoup plus de requêtes au total, soit environ 6734 requêtes contre 2153 au test précédent, ce qui représente une augmentation d'environ 213%.

Par contre, la fiabilité se détériore. Les endpoints de rapports avaient presque 0% d'échec avant Nginx, alors qu'ils montent à environ 35–37% d'échec avec Nginx. Pour POST /orders, le taux d'échec reste à 100%, donc le load balancing ne règle pas le problème principal.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders	2164	2164	18	84	60000	915.05	0	60016	149.92	29.9	29.9
GET	/orders/reports/best-sellers	2239	796	120	260	420	105.67	1	811	305.88	32.6	0
GET	/orders/reports/highest-spenders	2331	867	120	260	430	103.43	1	806	321.55	29.2	0
	Aggregated	6734	3827	47	240	470	364.99	0	60016	261.18	91.7	29.9

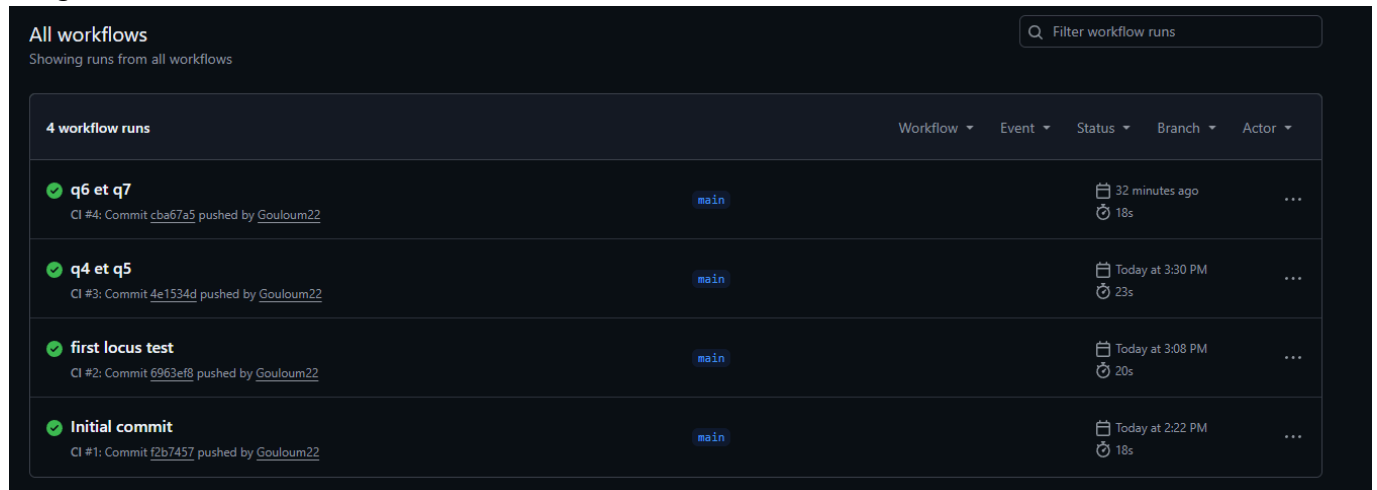
8.1 Capture d'écran des statistiques

Question 9 : Dans le fichier nginx.conf, il existe un attribut qui configure l'équilibrage de charge. Quelle politique d'équilibrage de charge utilisons-nous actuellement ? Consultez la documentation officielle de Nginx si vous avez des questions.

La politique d'équilibrage de charge utilisée est least_conn, c'est-à-dire la stratégie des moindres connexions. Avec cette stratégie, Nginx envoie les nouvelles requêtes vers l'instance backend qui a le moins de connexions actives au moment de la requête.

CI/CD

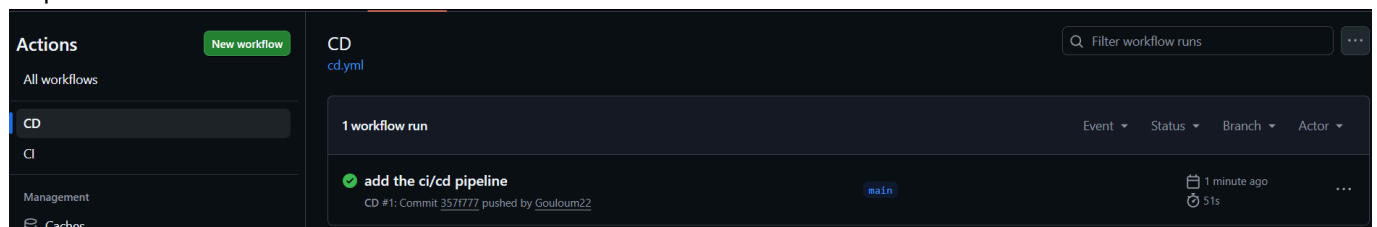
Intégration continue avec les tests:



The screenshot shows the 'All workflows' page in GitHub Actions. It displays a list of 4 workflow runs. Each run includes a status icon (green checkmark), a title, a commit hash and author, a branch name, and a completion time. The runs are: 'q6 et q7' (CI #4), 'q4 et q5' (CI #3), 'first locus test' (CI #2), and 'Initial commit' (CI #1).

Workflow	Event	Status	Branch	Actor
q6 et q7	CI #4: Commit cba67a5 pushed by Gouloum22	main	32 minutes ago	...
q4 et q5	CI #3: Commit 4e1534d pushed by Gouloum22	main	Today at 3:30 PM	...
first locus test	CI #2: Commit 6963ef8 pushed by Gouloum22	main	Today at 3:08 PM	...
Initial commit	CI #1: Commit f2b7457 pushed by Gouloum22	main	Today at 2:22 PM	...

Déploiement continue sur la VM:



The screenshot shows the 'Actions' page in GitHub, specifically the 'CD' workflow. It displays a single workflow run titled 'add the ci/cd pipeline' (CD #1) with a status of 'Succeeded'. The run was triggered by a commit push and completed 1 minute ago.

Workflow	Event	Status	Branch	Actor
add the ci/cd pipeline	CD #1: Commit 3571777 pushed by Gouloum22	main	1 minute ago	...

```
✓ Connected to GitHub

Current runner version: '2.334.0'
2026-06-10 21:09:47Z: Listening for Jobs
2026-06-10 21:12:07Z: Running job: deploy
2026-06-10 21:12:54Z: Job deploy completed with result: Succeeded
```